

How I Used Hindsight to Detect When Engineering Decisions Drift

A line like `RATE_LIMIT = 100` is easy to understand syntactically and surprisingly hard to understand historically. The difficult question is usually not what the code does, but why someone decided it should do that in the first place—and whether that reason still holds.

I built Archaeologist around that problem: persistent memory for engineering decisions. An engineer records what was decided, why, who made the decision, and when. Later, another engineer can ask why a particular value or workaround exists and get an answer grounded in the recorded history rather than an LLM's guess.

The interesting part is what happens when that history changes.

The problem is not missing documentation

Engineering systems accumulate decisions faster than they accumulate durable explanations.

A rate limit gets introduced because a downstream service became unstable. A timeout is increased after a production incident. A workaround is added because an upstream API behaves incorrectly. Months later, the surrounding system changes, but the original code often remains untouched.

The code still tells me the current state:

```
RATE_LIMIT = 100
```

It does not tell me that the limit was introduced because Elasticsearch became unstable above 100 requests per second. It also does not tell me that Elasticsearch was later upgraded and that the original constraint was considered resolved.

That distinction is what I wanted Archaeologist to preserve.

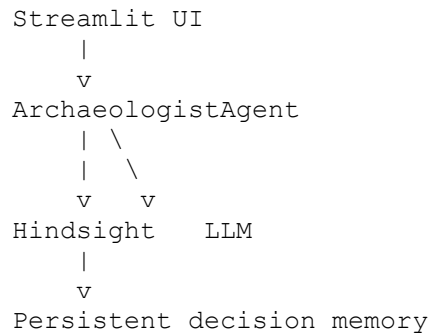
I deliberately did not build this as a documentation search system. The important information is temporal and attributable: what happened first, what changed later, and whether the later information affects the original reasoning.

How Archaeologist fits together

The application is intentionally small.

The Streamlit UI handles the interaction: Ask, Record, Update, and Timeline. The ArchaeologistAgent coordinates question parsing, memory retrieval, synthesis, timeline construction, and drift assessment. A dedicated memory layer is the only part of the application that talks to [Hindsight](#), while the LLM layer handles synthesis.

The architecture is essentially:



There is no application database sitting beside Hindsight and no separate vector store to keep synchronized. Hindsight is the persistence layer.

That separation turned out to be useful. The agent does not need to know how Hindsight stores memories, and the UI does not need to know how retrieval works. The memory layer translates application-level `DecisionRecord` objects into Hindsight `RETAIN` operations and maps `RECALL` results back into application models.

Why Hindsight is central to the design

I used the [Hindsight GitHub repository](#) and its [Hindsight documentation](#) as the foundation for the memory layer.

The important primitives are `RETAIN` and `RECALL`.

When an engineer records a decision, Archaeologist retains a structured narrative containing the entity, decision, reason, author, date, and optional evidence. The record also carries metadata and entity information so that later retrieval can be scoped to the code identifier.

The application uses the decision's real-world timestamp rather than treating the time of the database write as the event date. That matters because the whole point is reconstructing what happened over time.

One implementation detail became especially important: every retained decision gets a fresh document ID.

```
entity:search_api.rate_limit
kind: original
author: Priya Raman
date: 2023-04-11
```

An update is another document rather than an edit to the first one. That preserves the history instead of turning the memory store into a table containing only the latest state.

This is also where Hindsight's document semantics changed how I thought about the write path. If I reused a document ID, I could replace the previous document. For a system whose purpose is preserving engineering history, that is exactly the behavior I wanted to avoid.

The result is an additive history:

```
Original decision
|
+---- Later update
|
+---- Later update
|
+---- Another observation
```

The application can then RECALL that history later.

RECALL is not the final answer

A question such as:

Why is `search_api.rate_limit` still 100?

does not go directly from the UI to an LLM.

First, Archaeologist identifies the entity and retrieves relevant Hindsight memories. For an entity-scoped question, it combines broad semantic retrieval with an entity-specific tag query. The retrieved memories are then mapped into application models.

The next important decision was not to let the LLM determine chronology. The timeline is computed in code by sorting the retrieved memories using their real-world event date, falling back to the mentioned date when necessary. The model is then given that established history to explain.

That makes the boundary between retrieval and reasoning much clearer:

```
Hindsight
|
| What was actually recorded?
v
Retrieved memories
|
| Deterministic ordering
v
Chronological history
|
| LLM synthesis
v
Human-readable explanation
```

The LLM is the narrator, not the historical database.

I also treat retrieved memory as untrusted input. The synthesis step is instructed to stay grounded in the retrieved evidence, and generated citation IDs are checked against the memories that were actually returned. A model cannot simply invent a memory reference and have it appear in the UI.

This is one of the places where using [persistent agent memory](#) is different from asking a general-purpose model the same question without context. The model has a bounded historical record to reason over.

The interesting part: detecting decision drift

The central behavior of Archaeologist is decision drift.

Consider the history of a rate limit.

In April 2023, Priya Raman records:

search_api.rate_limit is 100 requests/sec because Elasticsearch became unstable above that level.

In August 2024, Marcus Webb records:

Elasticsearch was upgraded to a larger cluster and the original stability constraint was resolved in testing. The rate limit itself was not changed.

Now a developer asks:

Why is search_api.rate_limit still 100?

A normal code assistant can see the value 100. A documentation search can find a sentence explaining the original reason. Neither necessarily connects the two points in time.

Archaeologist recalls both memories.

The resulting history is:

```
2023-04-11
Rate limit = 100 requests/sec
Reason: Elasticsearch instability
Author: Priya Raman
  |
  v
2024-08-02
Elasticsearch upgraded
Original constraint resolved
Rate limit unchanged
Author: Marcus Webb
  |
  v
Current question
Why is the limit still 100?
```

The system can then classify the recorded evidence as may_be_outdated.

That wording is deliberate.

Archaeologist does not say:

The code is wrong.

It says, in effect:

The original reason appears to have changed, but the record does not show that the decision itself was revisited.

That is a much safer engineering signal.

The drift assessment has four states:

- no_evidence
- likely_current
- needs_review
- may_be_outdated

This keeps the system from turning incomplete historical evidence into a false certainty.

The cold-start case matters too

One of the simplest behaviors is also one of the most important.

If nobody has ever recorded a decision, Archaeologist should not manufacture an explanation.

For the primary example, `search_api.rate_limit` is deliberately absent from the initial seed data. The first question therefore produces an honest response:

I don't have recorded historical context for this decision.

Only after the engineer records the original decision does it become part of persistent memory.

That gives the system a clean progression:

```
No history
  ↓
RETAIN original decision
  ↓
History exists
  ↓
RETAIN later update
  ↓
RECALL accumulated history
  ↓
Detect possible drift
```

Without that cold-start state, it would be difficult to tell whether the system actually remembers something or is simply producing a plausible answer.

The implementation is intentionally conservative

I made several choices specifically to keep the historical behavior deterministic.

First, updates are additive. I never overwrite the original decision.

Second, timelines are sorted in code rather than generated by the LLM.

Third, retrieved memory is treated as evidence, not as instructions that can override the application's rules.

Fourth, conflicts are surfaced instead of silently resolved. If two memories disagree, the application keeps both sides with their authors and dates rather than asking the model to choose which engineer was "right."

Finally, drift is advisory. The system is designed to tell an engineer where the recorded reasoning deserves another look, not to automatically change production code.

The underlying idea is closely related to what [agent memory](#) means in persistent AI systems: the useful state is not just the current conversation. It is information that persists, accumulates, and can influence later reasoning.

What I learned building it

1. History needs its own data model

A current configuration value and the reason it was introduced are different kinds of information.

Once I modeled decisions explicitly—with an entity, decision, reason, author, date, and optional evidence—the rest of the system became easier to reason about. I was no longer trying to reconstruct history from arbitrary text.

2. Never make the LLM responsible for deterministic work

Chronology is a sorting problem.

Citation validation is a validation problem.

Whether a memory was actually retrieved is a retrieval problem.

The LLM is useful for explaining the evidence and assessing whether the recorded history suggests drift. Giving it responsibility for everything makes the system harder to trust.

3. Persistent memory changes the product, not just the implementation

Hindsight is not an infrastructure detail that could be swapped out without changing Archaeologist's behavior.

RETAIN gives the system accumulating history. RECALL gives it access to that history later. Without persistence, there is no meaningful timeline and no way to compare an old reason with a later observation.

4. "I don't know" is part of the feature

An engineering memory system should distinguish between:

- something that was recorded,
- something that can be inferred from recorded evidence,
- and something that was never recorded.

That third state is valuable. It prevents the system from turning missing history into invented history.

5. Decision drift is really about context, not code

The most useful output is not "change this value."

It is:

Here's why this value existed, here's what changed later, and here's why the original reasoning may deserve review.

That leaves the engineering decision with the person who owns the system while giving them the historical context they would otherwise have to reconstruct manually.

Closing

The hardest part of maintaining a large codebase is often not understanding what the code does. It is understanding why someone made a particular choice months or years ago.

Archaeologist treats that reasoning as persistent engineering memory.

An engineer records a decision. Hindsight retains it. Another engineer records what changed. Later, a question about an old value can retrieve the accumulated history, reconstruct its timeline, and identify where the original reasoning may have drifted.

That is the behavior I wanted from the system: not an AI that confidently explains every line of code, but one that knows the difference between what the team actually recorded and what it can only guess.